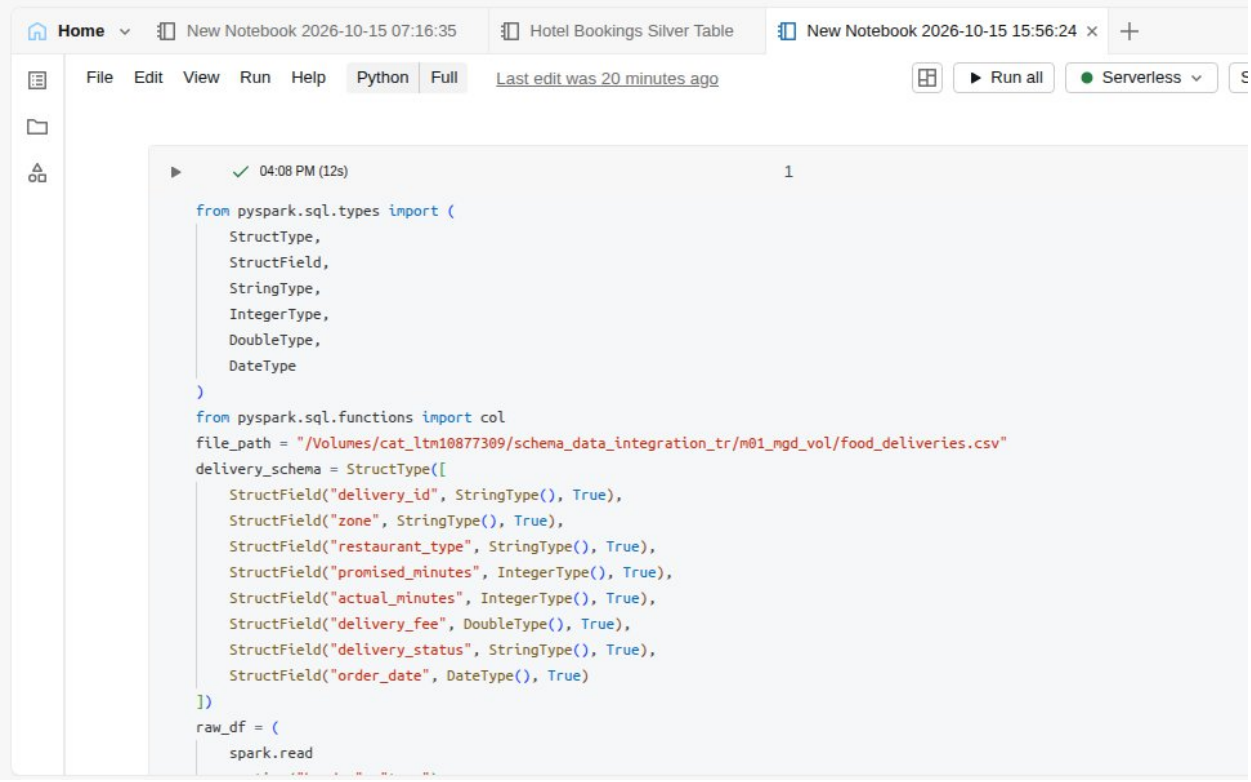
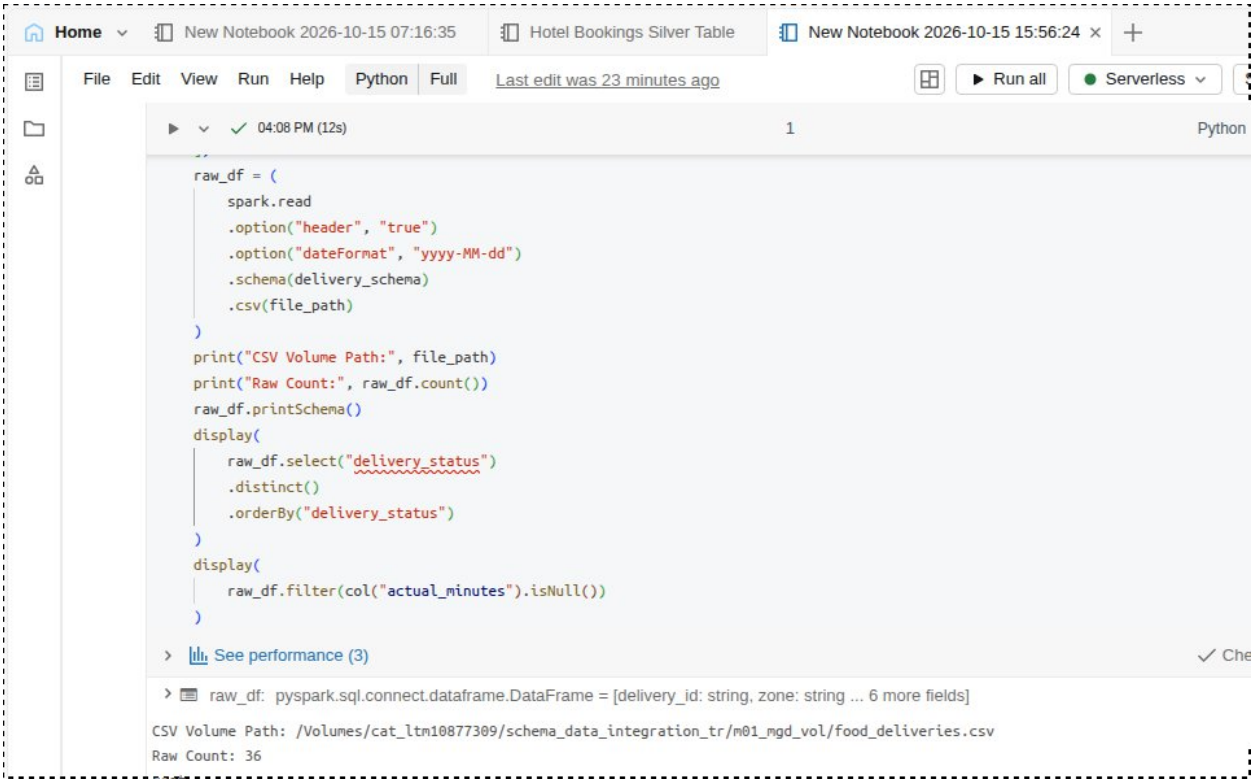
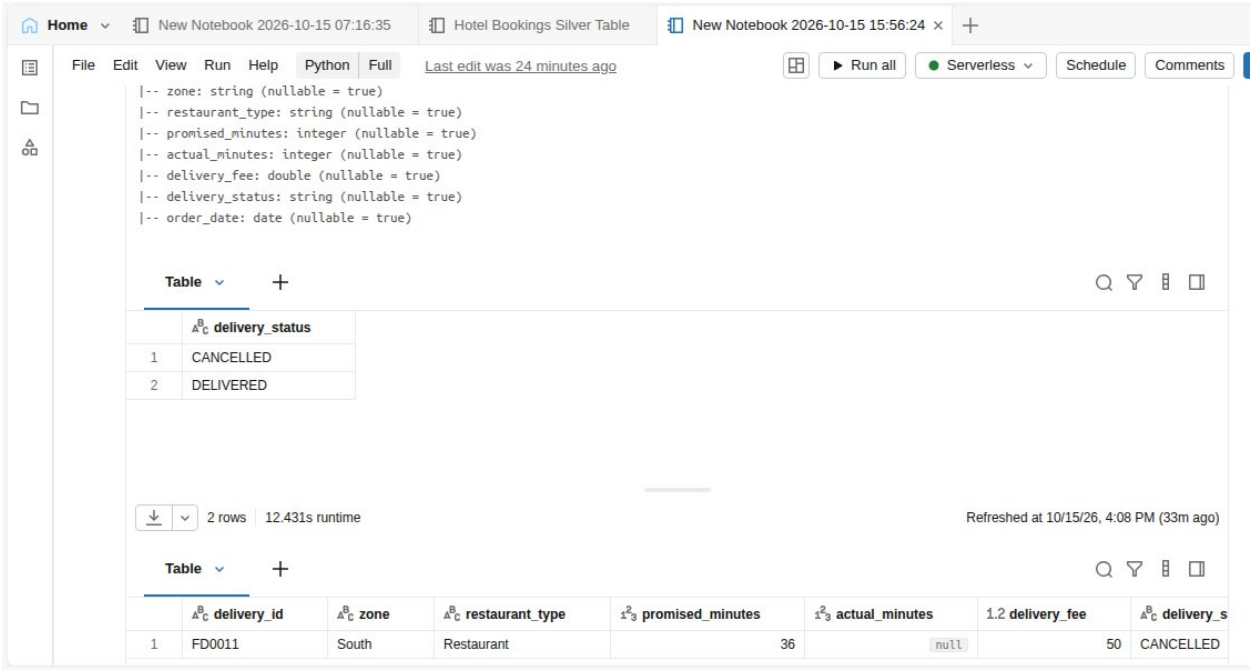
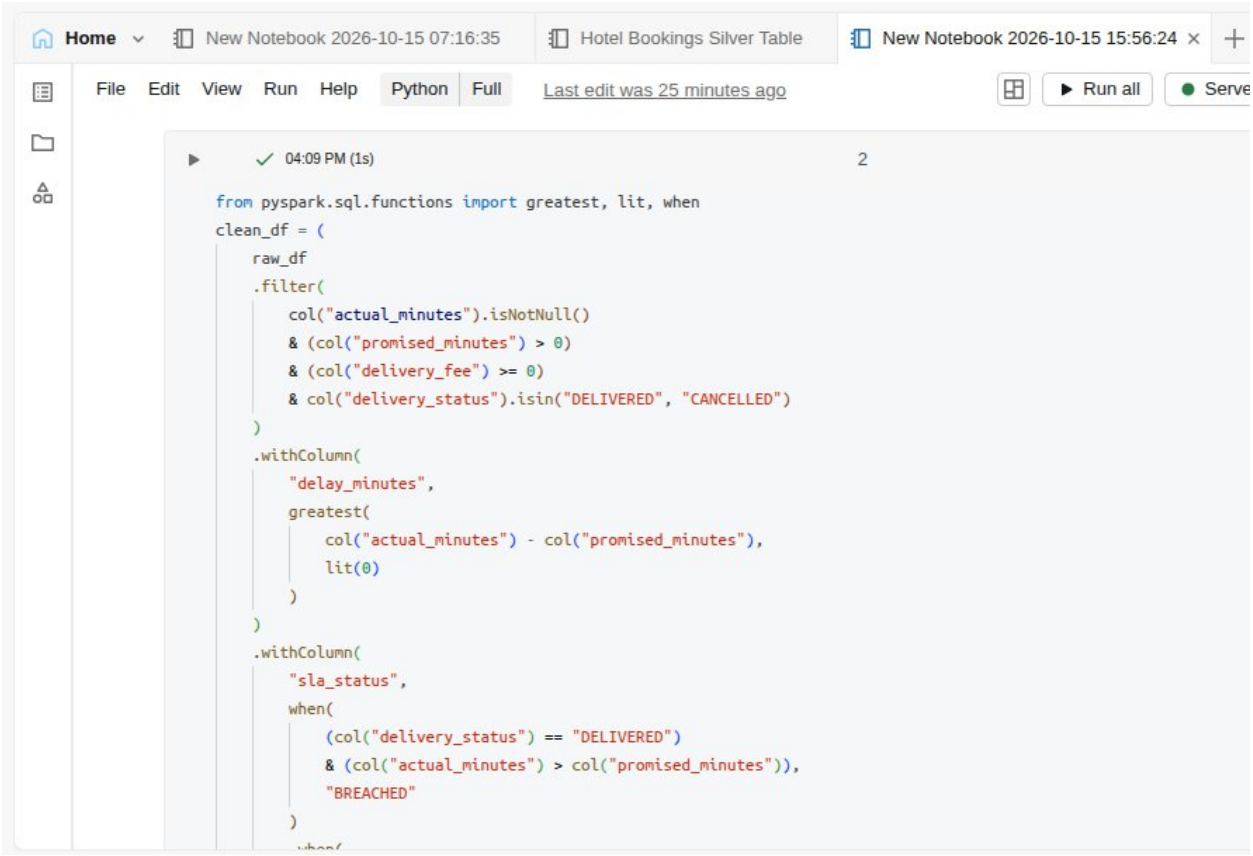
WEEK 9 DAY4 COD ASSIGNMENT TASK OUTPUTS :

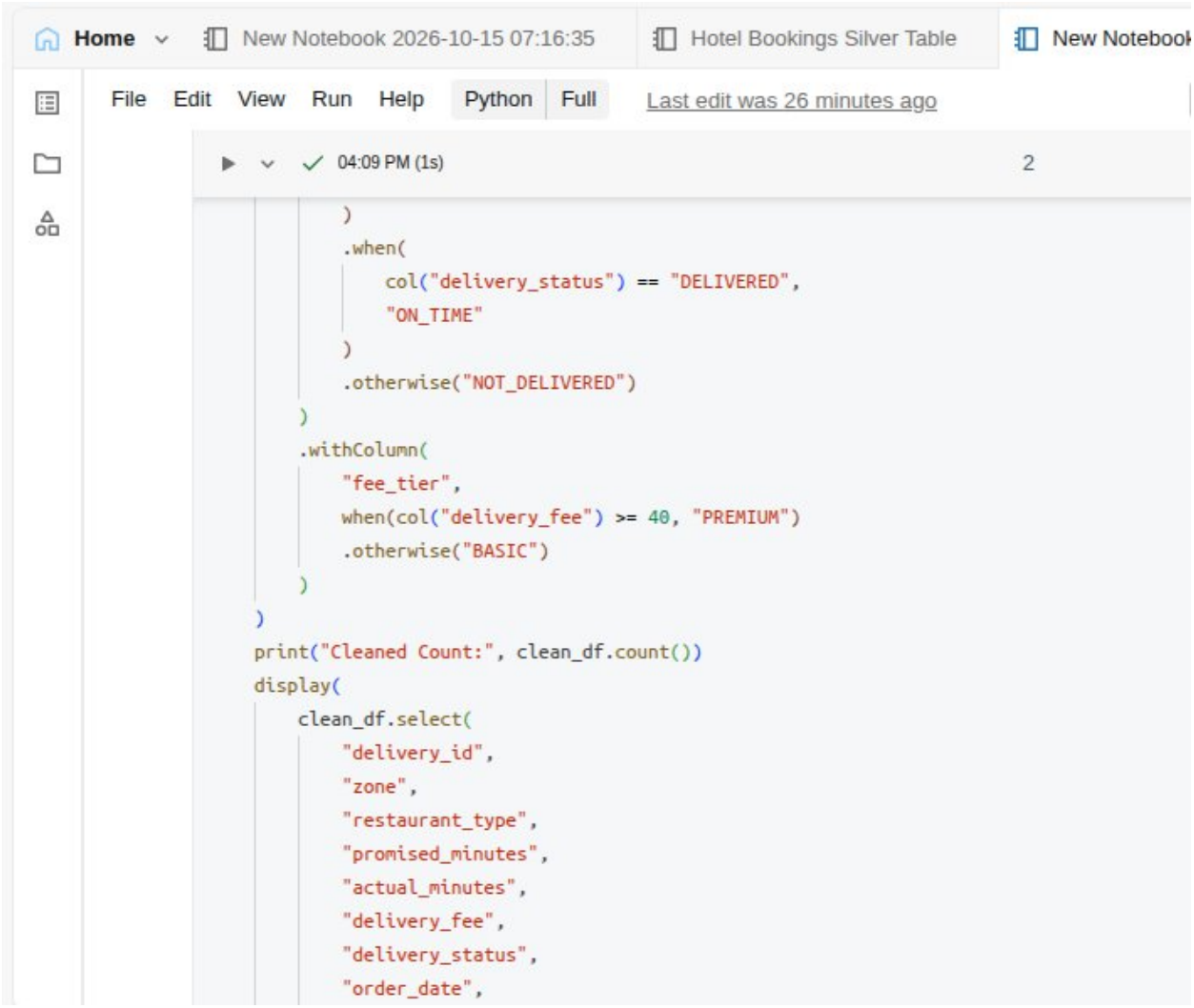
MEDIUM : FOOD DELIVERY

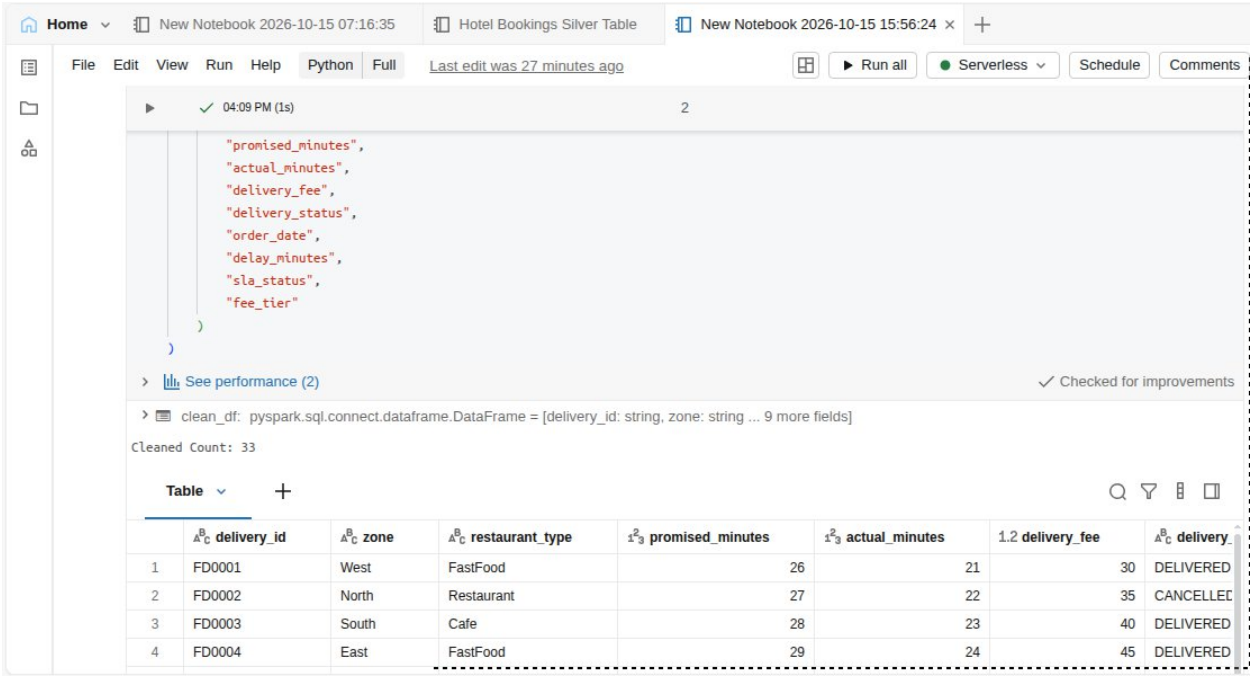


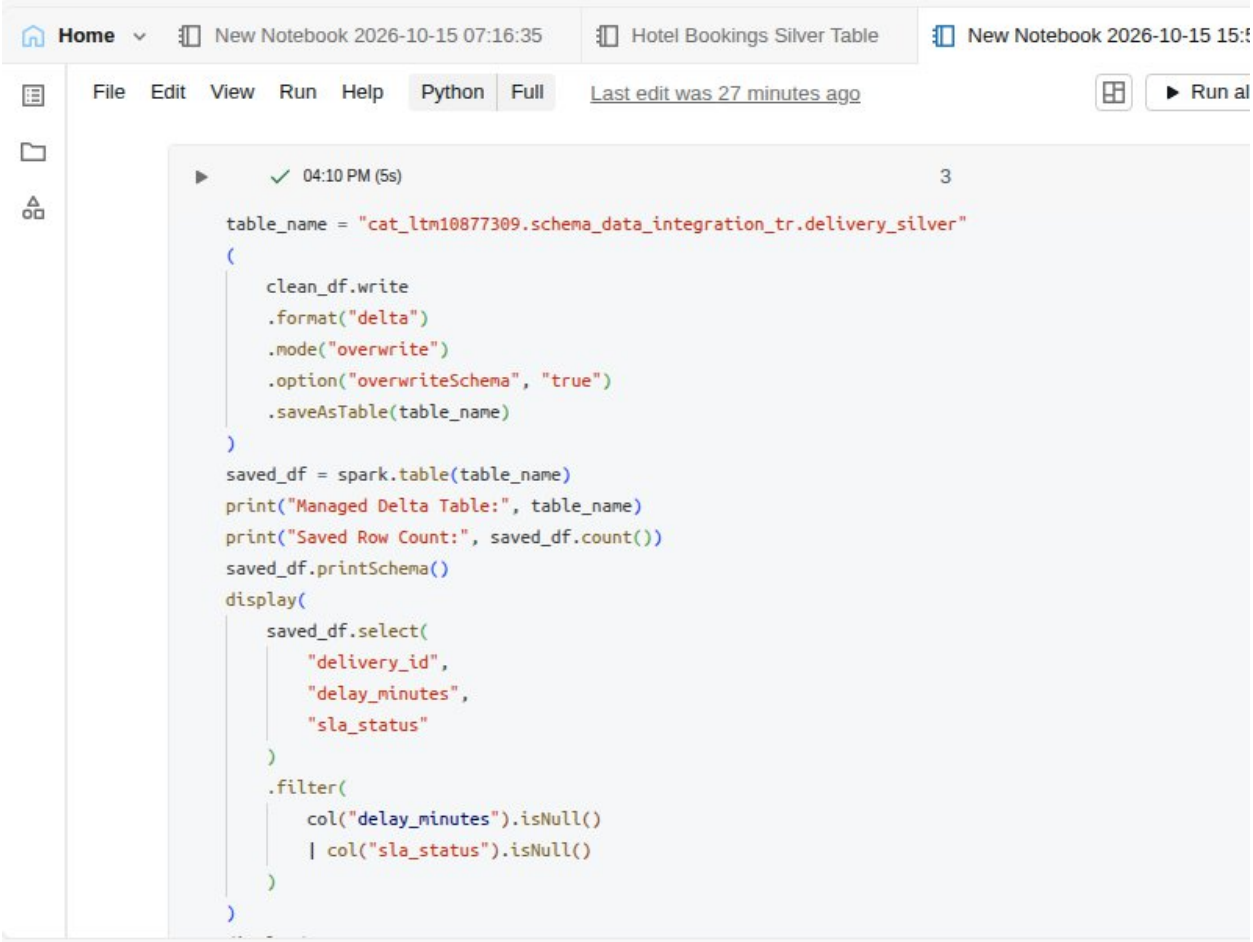


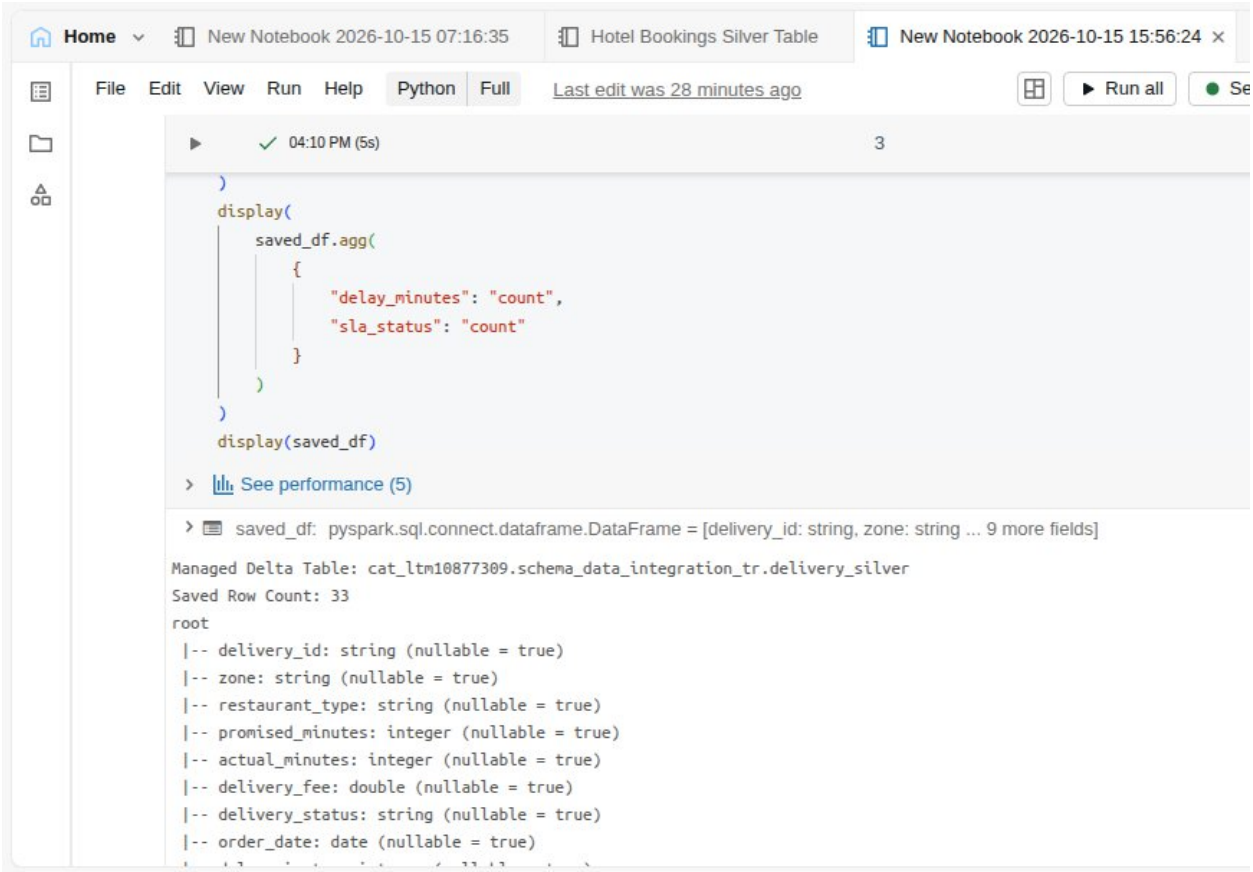

Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:56:24
File Edit View Run Help Python Full
Last edit was 24 minutes ago
Run all
Serverless
Schedule
Comments
|-- zone: string (nullable = true)
|-- restaurant_type: string (nullable = true)
|-- promised_minutes: integer (nullable = true)
|-- actual_minutes: integer (nullable = true)
|-- delivery_fee: double (nullable = true)
|-- delivery_status: string (nullable = true)
|-- order_date: date (nullable = true)
Table
delivery_status
1 CANCELLED
2 DELIVERED
2 rows | 12.431s runtime
Refreshed at 10/15/26, 4:08 PM (33m ago)
Table
delivery_id zone restaurant_type promised_minutes actual_minutes delivery_fee delivery_s
1 FD0011 South Restaurant 36 null 50 CANCELLED

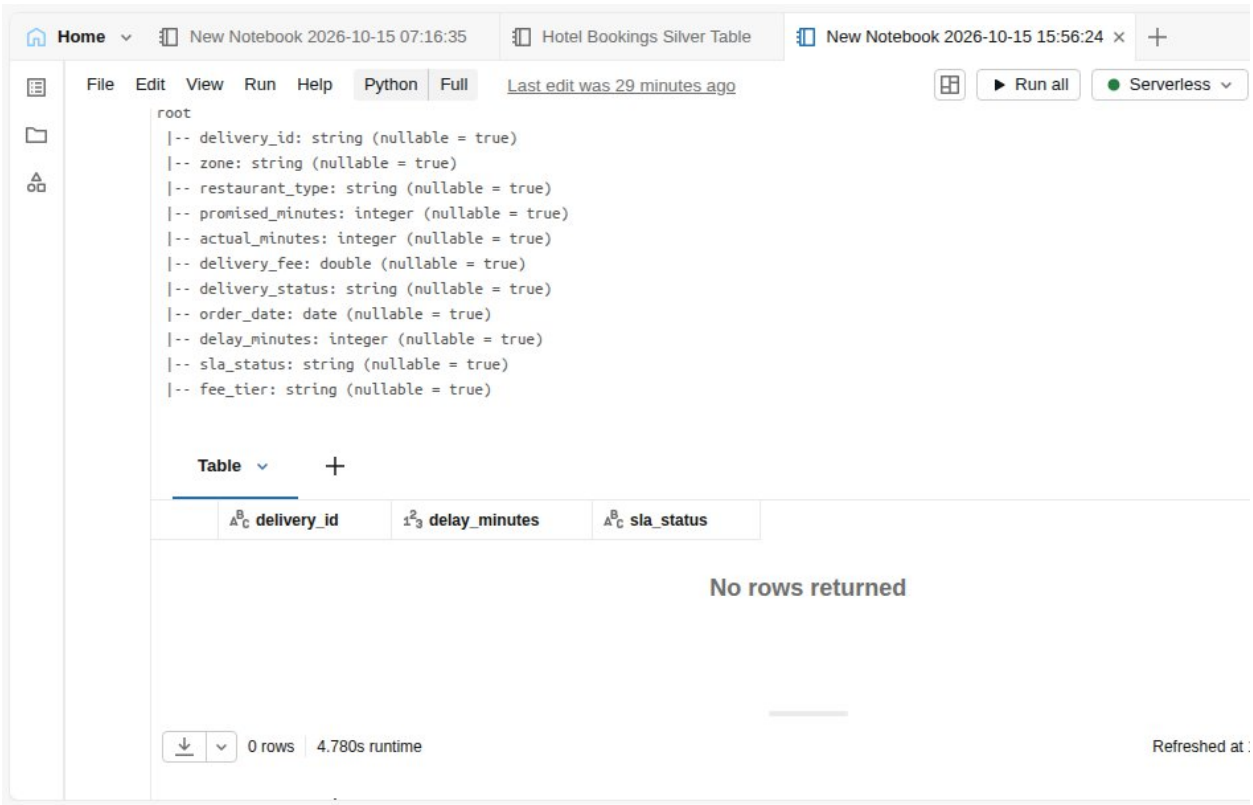

Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:56:24
File Edit View Run Help Python Full
Last edit was 25 minutes ago
Run all
04:09 PM (1s)
2
from pyspark.sql.functions import greatest, lit, when
clean_df = (
raw_df
.filter(
col("actual_minutes").isNotNull()
& (col("promised_minutes") > 0)
& (col("delivery_fee") >= 0)
& col("delivery_status").isin("DELIVERED", "CANCELLED")
)
.withColumn(
"delay_minutes",
greatest(
col("actual_minutes") - col("promised_minutes"),
lit(0)
)
)
.withColumn(
"sla_status",
when(
(col("delivery_status") == "DELIVERED")
& (col("actual_minutes") > col("promised_minutes")),
"BREACHED"
)

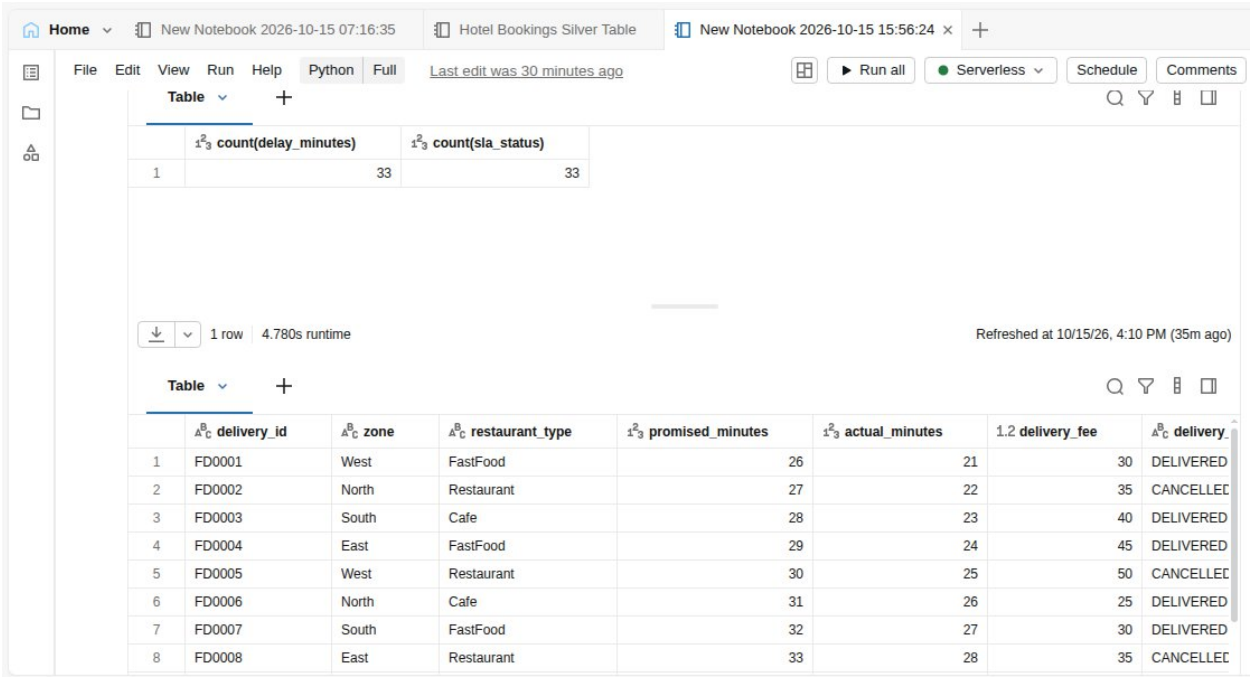

Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook
File Edit View Run Help Python Full
Last edit was 26 minutes ago
04:09 PM (1s)
2
)
.when(
col("delivery_status") == "DELIVERED",
"ON_TIME"
)
.otherwise("NOT_DELIVERED")
)
.withColumn(
"fee_tier",
when(col("delivery_fee") >= 40, "PREMIUM")
.otherwise("BASIC")
)
)
print("Cleaned Count:", clean_df.count())
display(
clean_df.select(
"delivery_id",
"zone",
"restaurant_type",
"promised_minutes",
"actual_minutes",
"delivery_fee",
"delivery_status",
"order_date",

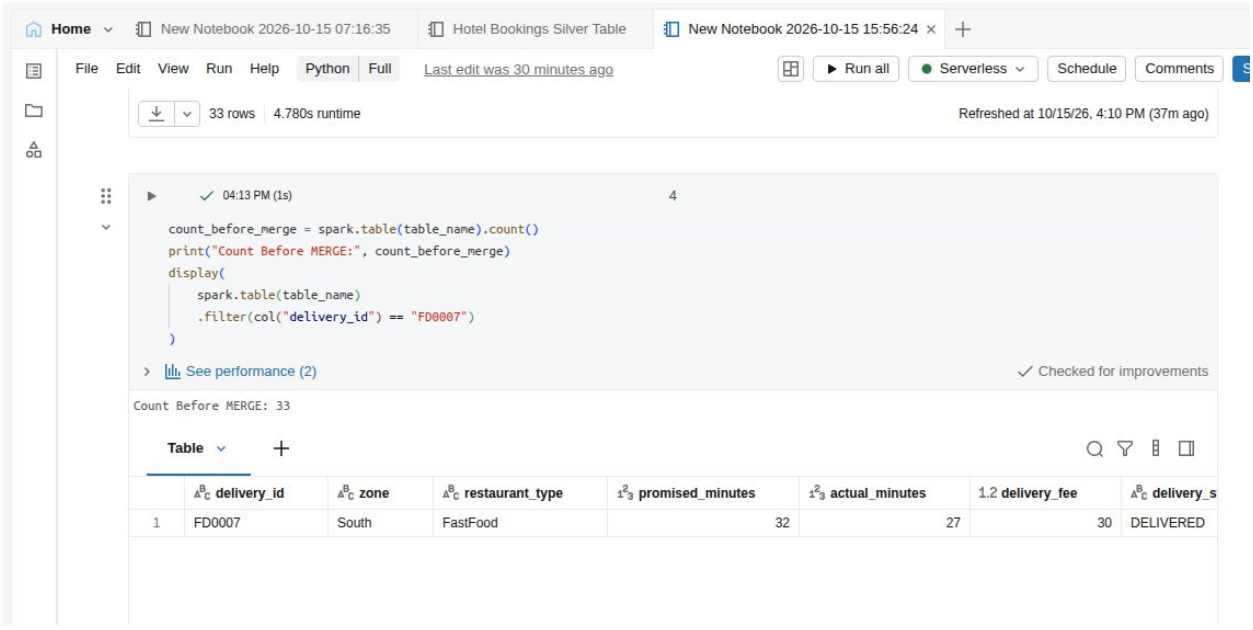

Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:56:24
File Edit View Run Help Python Full
Last edit was 27 minutes ago
Run all
Serverless
Schedule
Comments
04:09 PM (1s)
2
"promised_minutes",
"actual_minutes",
"delivery_fee",
"delivery_status",
"order_date",
"delay_minutes",
"sla_status",
"fee_tier"
See performance (2)
Checked for improvements
clean_df: pyspark.sql.connect.dataframe.DataFrame = [delivery_id: string, zone: string ... 9 more fields]
Cleaned Count: 33
Table
delivery_id zone restaurant_type promised_minutes actual_minutes delivery_fee delivery_
1 FD0001 West FastFood 26 21 30 DELIVERED
2 FD0002 North Restaurant 27 22 35 CANCELLED
3 FD0003 South Cafe 28 23 40 DELIVERED
4 FD0004 East FastFood 29 24 45 DELIVERED

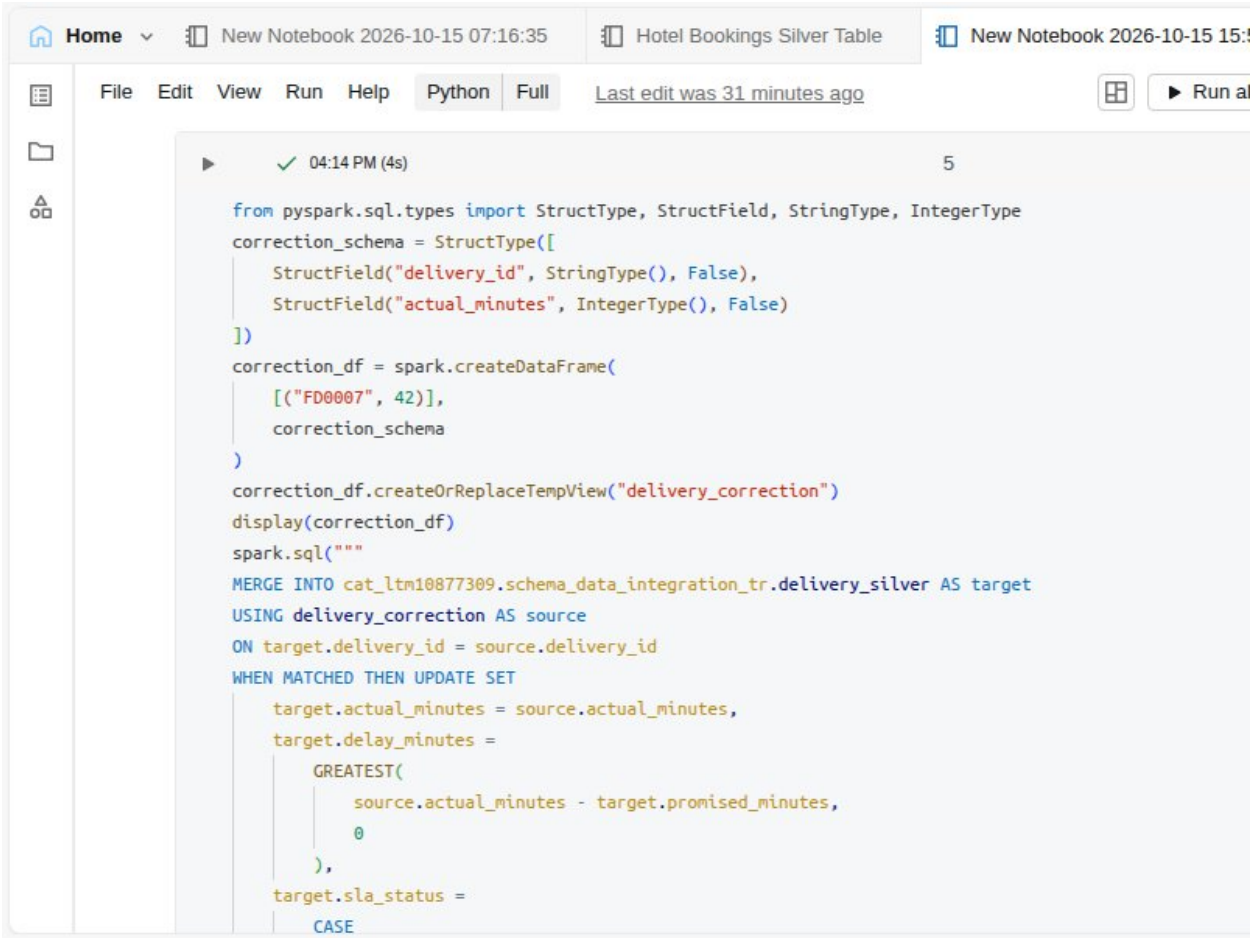

Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:5
File Edit View Run Help Python Full
Last edit was 27 minutes ago
Run al
04:10 PM (5s)
3
table_name = "cat_ltm10877309.schema_data_integration_tr.delivery_silver"
(
clean_df.write
.format("delta")
.mode("overwrite")
.option("overwriteSchema", "true")
.saveAsTable(table_name)
)
saved_df = spark.table(table_name)
print("Managed Delta Table:", table_name)
print("Saved Row Count:", saved_df.count())
saved_df.printSchema()
display(
saved_df.select(
"delivery_id",
"delay_minutes",
"sla_status"
)
.filter(
col("delay_minutes").isNull()
| col("sla_status").isNull()
)
)

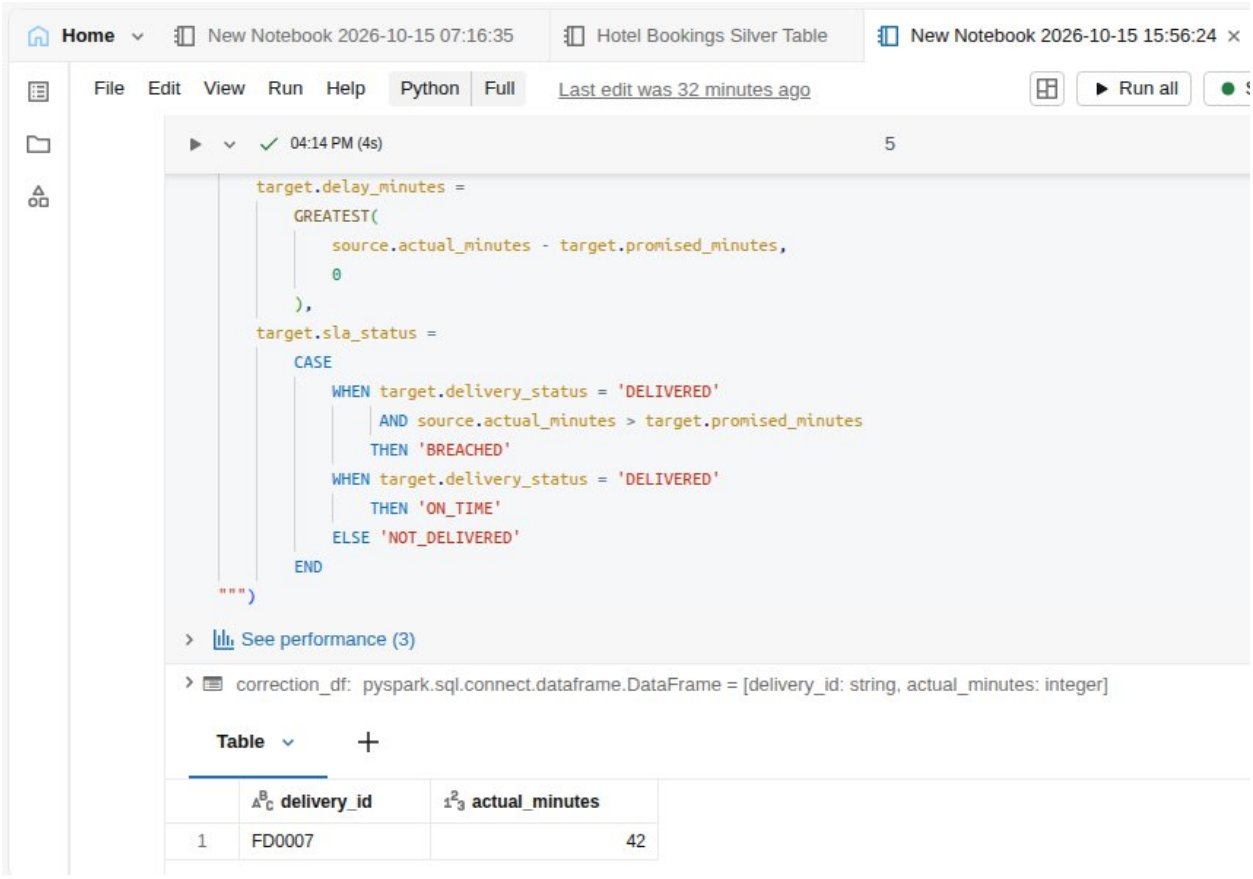

Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:56:24
File Edit View Run Help Python Full
Last edit was 28 minutes ago
Run all
04:10 PM (5s)
3
)
display(
saved_df.agg(
{
"delay_minutes": "count",
"sla_status": "count"
}
)
)
display(saved_df)
See performance (5)
saved_df: pyspark.sql.connect.dataframe.DataFrame = [delivery_id: string, zone: string ... 9 more fields]
Managed Delta Table: cat_ltm10877309.schema_data_integration_tr.delivery_silver
Saved Row Count: 33
root
|-- delivery_id: string (nullable = true)
|-- zone: string (nullable = true)
|-- restaurant_type: string (nullable = true)
|-- promised_minutes: integer (nullable = true)
|-- actual_minutes: integer (nullable = true)
|-- delivery_fee: double (nullable = true)
|-- delivery_status: string (nullable = true)
|-- order_date: date (nullable = true)

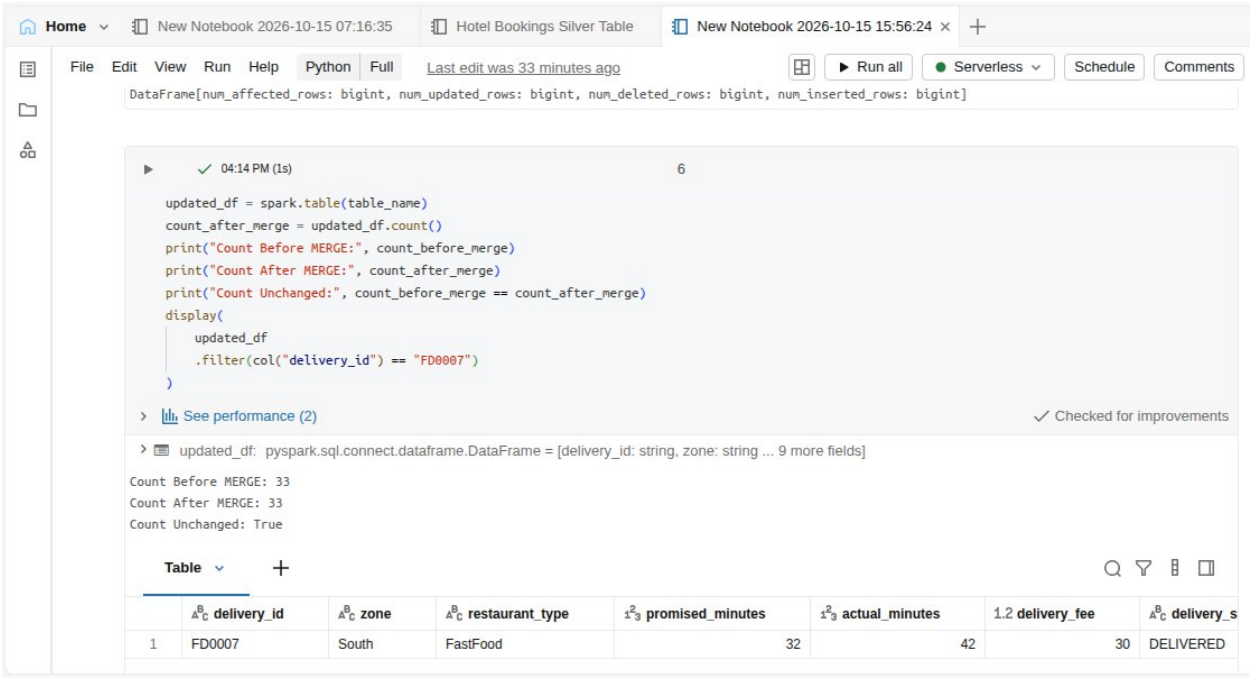

Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:56:24
File Edit View Run Help Python Full
Last edit was 29 minutes ago
Run all
Serverless
root
|-- delivery_id: string (nullable = true)
|-- zone: string (nullable = true)
|-- restaurant_type: string (nullable = true)
|-- promised_minutes: integer (nullable = true)
|-- actual_minutes: integer (nullable = true)
|-- delivery_fee: double (nullable = true)
|-- delivery_status: string (nullable = true)
|-- order_date: date (nullable = true)
|-- delay_minutes: integer (nullable = true)
|-- sla_status: string (nullable = true)
|-- fee_tier: string (nullable = true)
Table
delivery_id
delay_minutes
sla_status
No rows returned
0 rows
4.780s runtime
Refreshed at

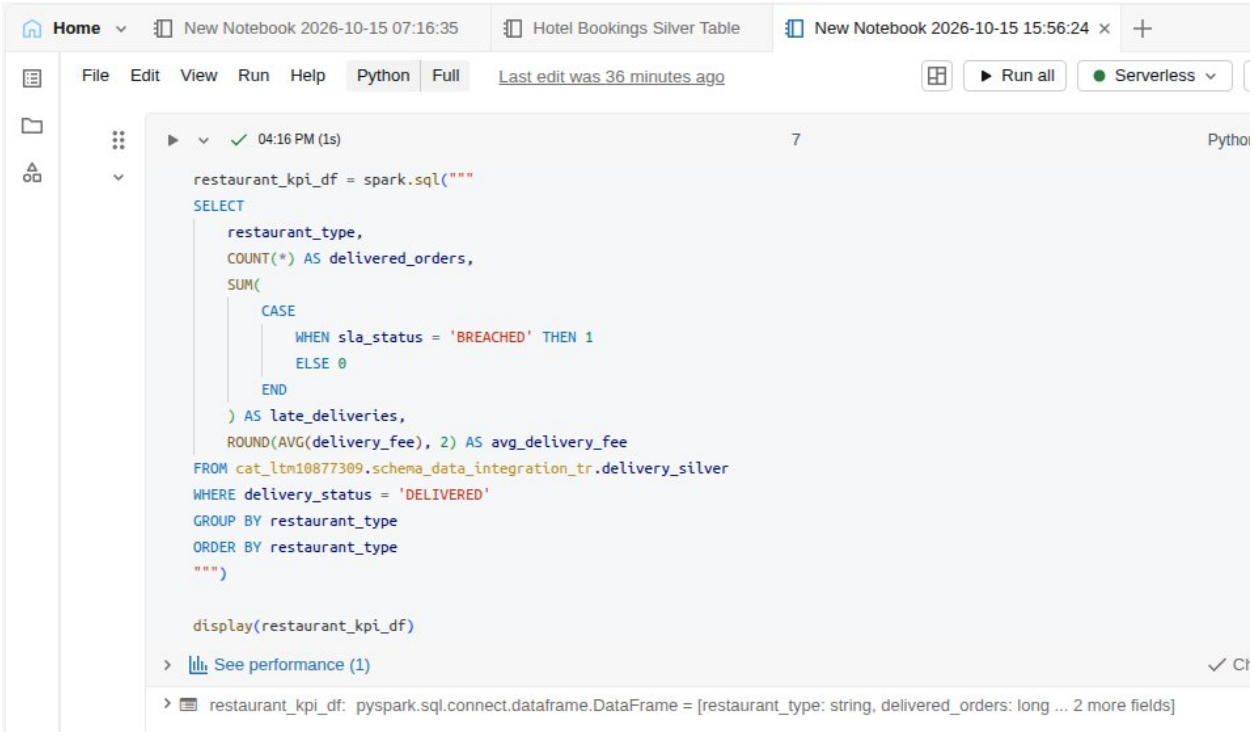

Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:56:24
File Edit View Run Help Python Full
Last edit was 30 minutes ago
Run all
Serverless
Schedule
Comments
Table
count(delay_minutes)
count(sla_status)
1 33 33
1 row | 4.780s runtime
Refreshed at 10/15/26, 4:10 PM (35m ago)
Table
delivery_id zone restaurant_type promised_minutes actual_minutes delivery_fee delivery
1 FD0001 West FastFood 26 21 30 DELIVERED
2 FD0002 North Restaurant 27 22 35 CANCELLED
3 FD0003 South Cafe 28 23 40 DELIVERED
4 FD0004 East FastFood 29 24 45 DELIVERED
5 FD0005 West Restaurant 30 25 50 CANCELLED
6 FD0006 North Cafe 31 26 25 DELIVERED
7 FD0007 South FastFood 32 27 30 DELIVERED
8 FD0008 East Restaurant 33 28 35 CANCELLED

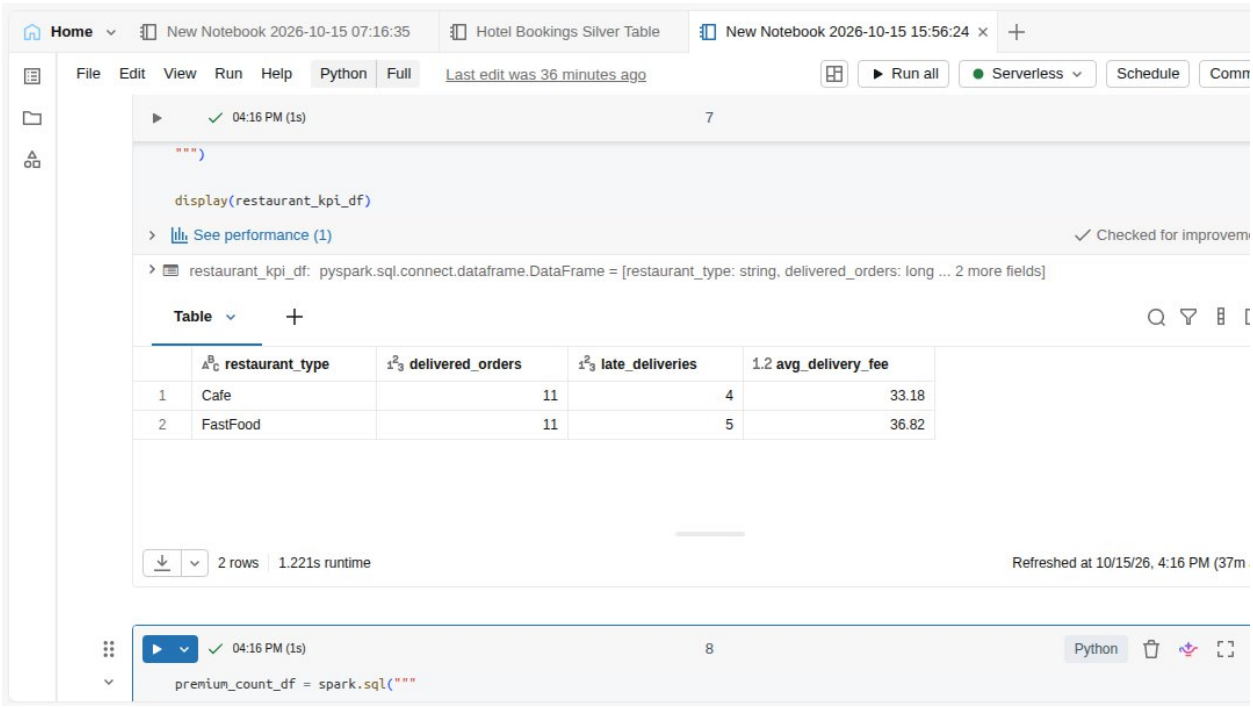

Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:56:24
File Edit View Run Help Python Full
Last edit was 30 minutes ago
Run all
Serverless
Schedule
Comments
33 rows | 4.780s runtime
Refreshed at 10/15/26, 4:10 PM (37m ago)
04:13 PM (1s)
4
count_before_merge = spark.table(table_name).count()
print("Count Before MERGE:", count_before_merge)
display(
spark.table(table_name)
.filter(col("delivery_id") == "FD0007")
)
See performance (2)
Checked for improvements
Count Before MERGE: 33
Table
delivery_id zone restaurant_type promised_minutes actual_minutes delivery_fee delivery_s
1 FD0007 South FastFood 32 27 30 DELIVERED

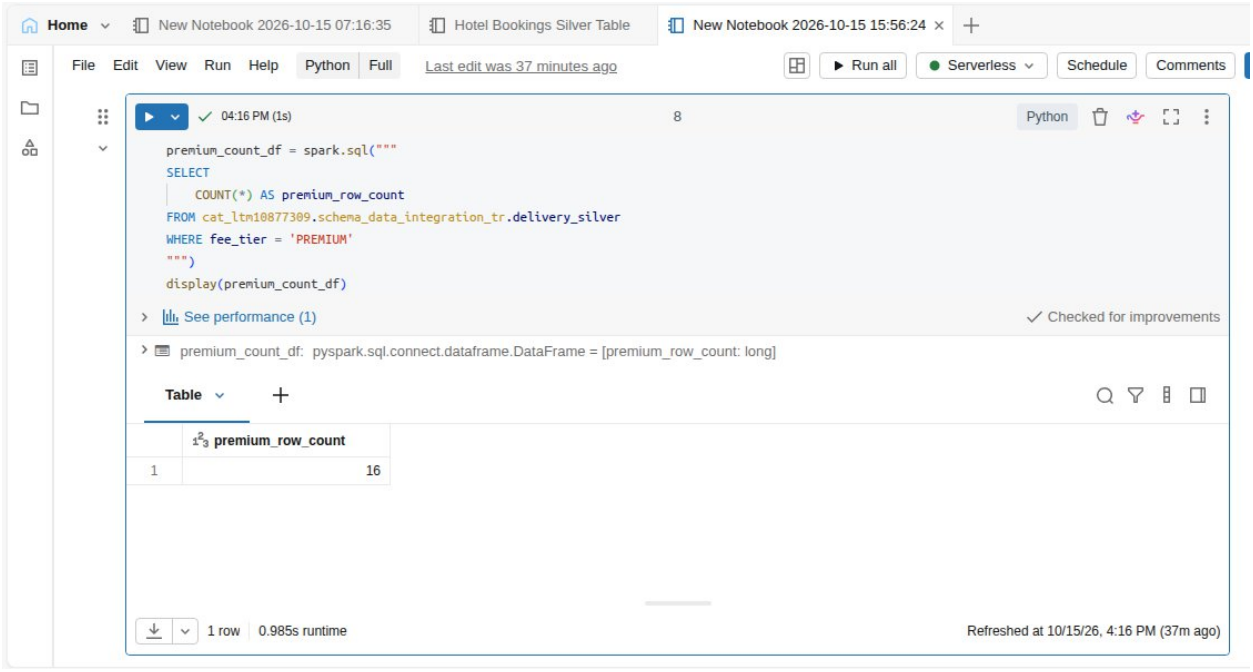

Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:
File Edit View Run Help Python Full
Last edit was 31 minutes ago
Run al
04:14 PM (4s)
5
from pyspark.sql.types import StructType, StructField, StringType, IntegerType
correction_schema = StructType([
StructField("delivery_id", StringType(), False),
StructField("actual_minutes", IntegerType(), False)
])
correction_df = spark.createDataFrame(
[("FD0007", 42)],
correction_schema
)
correction_df.createOrReplaceTempView("delivery_correction")
display(correction_df)
spark.sql("""
MERGE INTO cat_ltm10877309.schema_data_integration_tr.delivery_silver AS target
USING delivery_correction AS source
ON target.delivery_id = source.delivery_id
WHEN MATCHED THEN UPDATE SET
target.actual_minutes = source.actual_minutes,
target.delay_minutes =
GREATEST(
source.actual_minutes - target.promised_minutes,
0
),
target.sla_status =
CASE


Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:56:24
File Edit View Run Help Python Full
Last edit was 32 minutes ago
Run all
04:14 PM (4s)
5
target.delay_minutes =
GREATEST(
source.actual_minutes - target.promised_minutes,
0
),
target.sla_status =
CASE
WHEN target.delivery_status = 'DELIVERED'
AND source.actual_minutes > target.promised_minutes
THEN 'BREACHED'
WHEN target.delivery_status = 'DELIVERED'
THEN 'ON_TIME'
ELSE 'NOT_DELIVERED'
END
""")
See performance (3)
correction_df: pyspark.sql.connect.dataframe.DataFrame = [delivery_id: string, actual_minutes: integer]
Table
delivery_id
actual_minutes
1
FD0007
42


Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:56:24
File Edit View Run Help Python Full
Last edit was 33 minutes ago
Run all
Serverless
Schedule
Comments
DataFrame[num_affected_rows: bigint, num_updated_rows: bigint, num_deleted_rows: bigint, num_inserted_rows: bigint]
04:14 PM (1s)
6
updated_df = spark.table(table_name)
count_after_merge = updated_df.count()
print("Count Before MERGE:", count_before_merge)
print("Count After MERGE:", count_after_merge)
print("Count Unchanged:", count_before_merge == count_after_merge)
display(
updated_df
.filter(col("delivery_id") == "FD0007")
)
See performance (2)
Checked for improvements
updated_df: pyspark.sql.connect.dataframe.DataFrame = [delivery_id: string, zone: string ... 9 more fields]
Count Before MERGE: 33
Count After MERGE: 33
Count Unchanged: True
Table
delivery_id
zone
restaurant_type
promised_minutes
actual_minutes
delivery_fee
delivery_s
1
FD0007
South
FastFood
32
42
30
DELIVERED


Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:56:24
File Edit View Run Help Python Full
Last edit was 36 minutes ago
Run all
Serverless
04:16 PM (1s)
7
restaurant_kpi_df = spark.sql("""
SELECT
restaurant_type,
COUNT(*) AS delivered_orders,
SUM(
CASE
WHEN sla_status = 'BREACHED' THEN 1
ELSE 0
END
) AS late_deliveries,
ROUND(AVG(delivery_fee), 2) AS avg_delivery_fee
FROM cat_ltm10877309.schema_data_integration_tr.delivery_silver
WHERE delivery_status = 'DELIVERED'
GROUP BY restaurant_type
ORDER BY restaurant_type
""")
display(restaurant_kpi_df)
See performance (1)
restaurant_kpi_df: pyspark.sql.connect.dataframe.DataFrame = [restaurant_type: string, delivered_orders: long ... 2 more fields]


Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:56:24
File Edit View Run Help Python Full
Last edit was 36 minutes ago
Run all
Serverless
Schedule
04:16 PM (1s)
7
""")
display(restaurant_kpi_df)
See performance (1)
Checked for improvem
restaurant_kpi_df: pyspark.sql.connect.dataframe.DataFrame = [restaurant_type: string, delivered_orders: long ... 2 more fields]
Table
restaurant_type
delivered_orders
late_deliveries
avg_delivery_fee
1 Cafe 11 4 33.18
2 FastFood 11 5 36.82
2 rows
1.221s runtime
Refreshed at 10/15/26, 4:16 PM (37m
04:16 PM (1s)
8
Python
premium_count_df = spark.sql("""


Home
New Notebook 2026-10-15 07:16:35
Hotel Bookings Silver Table
New Notebook 2026-10-15 15:56:24
File Edit View Run Help Python Full
Last edit was 37 minutes ago
Run all
Serverless
Schedule
Comments
04:16 PM (1s)
8
Python
premium_count_df = spark.sql("""
SELECT
COUNT(*) AS premium_row_count
FROM cat_ltm10877309.schema_data_integration_tr.delivery_silver
WHERE fee_tier = 'PREMIUM'
""")
display(premium_count_df)
See performance (1)
Checked for improvements
premium_count_df: pyspark.sql.connect.dataframe.DataFrame = [premium_row_count: long]
Table
premium_row_count
1
16
1 row
0.985s runtime
Refreshed at 10/15/26, 4:16 PM (37m ago)